Formation Vue.js Moderne

Jour 1 — Fondamentaux

Modules 1 à 4

JavaScript · Écosystème · Vite · Composants

📋 Sommaire — Jour 1

#	Module	Contenu	Slides
1	**JavaScript — les bases**	Variables, fonctions, DOM, callbacks	4–22
2	**Écosystème JS Moderne**	ECMAScript, TypeScript, DevTools	23–42
3	**Vite + Vue.js**	Setup, architecture, scripts	43–60
4	**Les Composants**	SFC, props, emit, cycle de vie	61–100

> 🕐 Durée estimée : 7h de formation

Module 1

JavaScript — Apprendre les bases du langage

Socle indispensable avant Vue.js

Module 1 — Plan

1. Introduction et contexte
2. Variables et types
3. Fonctions
4. Conditions et boucles
5. Objets et tableaux
6. Manipulation du DOM
7. Callbacks
8. ✏️ Exercice

1.1 — Pourquoi JavaScript ?

JavaScript est :

- Le seul langage natif du navigateur
- Exécuté côté client **ET** serveur (Node.js)
- La base de tous les frameworks modernes (Vue, React, Angular)
- Le langage le plus utilisé au monde (Stack Overflow 2023)

Notre parcours :

```
JavaScript
    ↓
TypeScript
    ↓
Vue.js 3
```

1.2 — Variables : `let` et `const`

```
// ✅ const — valeur non réassignable (à préférer)
const nom = "Alice"
const PI = 3.14159

// ✅ let — valeur réassignable
let score = 0
score = 42

// ❌ var — à éviter (portée fonction, hoisting)
var ancien = "ne plus utiliser"
```

Règle simple :

- Utilise `const` par défaut
- Utilise `let` seulement si tu dois réassigner
- N'utilise **jamais** `var`

1.3 — Types de données

```
// Primitifs
const texte    = "Bonjour"          // string
const nombre   = 42                  // number
const decimal  = 3.14                // number (pas de float séparé)
const vrai     = true                // boolean
const rien     = null                // null
const inconnu  = undefined           // undefined

// Complexes
const tableau  = [1, 2, 3]           // Array
const objet    = { nom: "Alice" }    // Object
const fn       = () => "salut"       // Function
```

```
// Vérifier un type
typeof texte    // "string"
typeof nombre   // "number"
Array.isArray(tableau)  // true
```

1.4 — Fonctions : 3 syntaxes

```
// 1. Déclaration classique (hoistée)
function addition(a, b) {
  return a + b
}

// 2. Expression (non hoistée)
const soustraction = function(a, b) {
  return a - b
}

// 3. Fléchée — syntaxe moderne ✅
const multiplication = (a, b) => a * b

// Avec corps de fonction si plusieurs instructions
const diviser = (a, b) => {
  if (b === 0) return null
  return a / b
}
```

1.5 — Paramètres par défaut et rest

```
// Valeur par défaut
function saluer(nom = "anonyme") {
  return `Bonjour ${nom} !`
}
saluer()          // "Bonjour anonyme !"
saluer("Alice")   // "Bonjour Alice !"

// Rest parameters — tableau des arguments restants
function somme(...nombres) {
  return nombres.reduce((acc, n) => acc + n, 0)
}
somme(1, 2, 3, 4)  // 10

// Déstructuration en paramètre
function afficherUser({ nom, age }) {
  return `${nom} a ${age} ans`
}
afficherUser({ nom: "Bob", age: 30 })
```

1.6 — Conditions

```
// if / else if / else
const note = 85
if (note >= 90) {
  console.log("Excellent")
} else if (note >= 70) {
  console.log("Bien")        // ← affiché
} else {
  console.log("Insuffisant")
}

// Opérateur ternaire
const mention = note >= 70 ? "Admis" : "Refusé"

// Nullish coalescing
const prenom = null ?? "Inconnu"   // "Inconnu"

// Optional chaining
const ville = user?.adresse?.ville  // undefined si user est null
```

1.7 — Boucles et itération

```
const fruits = ["pomme", "poire", "cerise"]

// for classique
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i])
}

// for...of — itération sur les valeurs ✅
for (const fruit of fruits) {
  console.log(fruit)
}

// Méthodes de tableau (à préférer)
fruits.forEach(f => console.log(f))

const majuscules = fruits.map(f => f.toUpperCase())
const longs      = fruits.filter(f => f.length > 5)
const total      = [1,2,3].reduce((acc, n) => acc + n, 0)
```

1.8 — Objets et déstructuration

```
const utilisateur = {
  nom: "Alice",
  age: 28,
  adresse: { ville: "Lyon", cp: "69001" }
}

// Accès
utilisateur.nom          // "Alice"
utilisateur["age"]       // 28

// Déstructuration ✅
const { nom, age } = utilisateur
const { adresse: { ville } } = utilisateur

// Spread
const copie = { ...utilisateur, age: 29 }

// Méthodes utiles
Object.keys(utilisateur)    // ["nom", "age", "adresse"]
Object.values(utilisateur)  // ["Alice", 28, {...}]
Object.entries(utilisateur) // [["nom","Alice"], ...]
```

1.9 — Tableaux — méthodes essentielles

```
const produits = [
  { id: 1, nom: "Clavier", prix: 49 },
  { id: 2, nom: "Souris",  prix: 29 },
  { id: 3, nom: "Écran",   prix: 299 },
]

// Trouver
produits.find(p => p.id === 2)          // { id: 2, ... }
produits.findIndex(p => p.prix > 100)  // 2

// Transformer
produits.map(p => p.nom)               // ["Clavier","Souris","Écran"]
produits.filter(p => p.prix < 100)     // [clavier, souris]

// Réduire
produits.reduce((sum, p) => sum + p.prix, 0)  // 377

// Trier
[...produits].sort((a, b) => a.prix - b.prix)
```

1.10 — Manipulation du DOM

```
// Sélectionner des éléments
const btn   = document.querySelector("#mon-btn")
const items = document.querySelectorAll(".item")

// Modifier
btn.textContent = "Nouveau texte"
btn.style.color = "red"
btn.classList.add("actif")
btn.classList.toggle("visible")

// Créer et insérer
const li = document.createElement("li")
li.textContent = "Nouveau item"
document.querySelector("ul").appendChild(li)

// Supprimer
li.remove()
```

1.11 — Événements

```
const btn = document.querySelector("#btn-ajouter")

// Écouter un événement
btn.addEventListener("click", (event) => {
  console.log("Cliqué !", event.target)
})

// Événements courants
// click, dblclick, mouseover, mouseleave
// keydown, keyup, keypress
// input, change, submit
// DOMContentLoaded, load

// Supprimer un écouteur
const handleClick = () => console.log("ok")
btn.addEventListener("click", handleClick)
btn.removeEventListener("click", handleClick)
```

1.12 — Callbacks et asynchrone

```
// Callback — fonction passée en argument
function chargerDonnees(url, callback) {
  // Simule un délai réseau
  setTimeout(() => {
    const data = { id: 1, nom: "Alice" }
    callback(data)
  }, 1000)
}

chargerDonnees("/api/user", (user) => {
  console.log("Reçu :", user.nom)
})

// Promises — syntaxe moderne
fetch("/api/users")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(err => console.error(err))

// async / await — encore plus lisible ✅
const users = await fetch("/api/users").then(r => r.json())
```

1.13 — Modules ES6

```
// utils.js — exporter
export const PI = 3.14159
export function addition(a, b) { return a + b }
export default class Calculatrice { ... }

// main.js — importer
import Calculatrice from './utils.js'         // défaut
import { PI, addition } from './utils.js'    // nommés
import * as utils from './utils.js'           // tout

// Renommer à l'import
import { addition as add } from './utils.js'
```

💡 Vue.js utilise ce système de modules en permanence : chaque `.vue` est un module.

1.14 — Template Literals & Spread

```
// Template literals
const nom = "Alice"
const age = 28
const msg = `Bonjour ${nom}, vous avez ${age} ans.`
const multiline = `
  Ligne 1
  Ligne 2
`

// Spread operator
const tab1 = [1, 2, 3]
const tab2 = [4, 5, 6]
const fusion = [...tab1, ...tab2]          // [1,2,3,4,5,6]

const obj1 = { a: 1 }
const obj2 = { b: 2 }
const merge = { ...obj1, ...obj2 }         // { a: 1, b: 2 }

// Clone (shallow)
const copie = [...tab1]
const objCopie = { ...obj1 }
```

1.15 — Récapitulatif Module 1

Concept	Syntaxe clé
Variables	`const` / `let`
Fonctions	`const fn = (a) => a * 2`
Conditions	`if/else`, opérateur `? :`
Boucles	`for...of`, `.map()`, `.filter()`
Objets	`{ ...obj }`, destructuration
DOM	`querySelector`, `addEventListener`
Async	`async/await`, `fetch`
Modules	`export` / `import`

✏️ Exercice — Module 1

Exercice 1.1 — Fonctions et conditions

Exercice 1.2 — Manipulation du DOM

Exercice 1.3 — ToDoList *(Projet fil rouge)*

> 🕐 Durée estimée : 45 minutes

Module 2

L'écosystème du développeur JS Moderne

ECMAScript · TypeScript · Tooling · DevTools

Module 2 — Plan

1. Features modernes d'ECMAScript
2. Introduction à TypeScript
3. Types de base et interfaces
4. Generics et utilitaires
5. Tooling moderne (npm, ESLint, Prettier)
6. Vue DevTools
7. Debugging
8. ✏️ Exercice

2.1 — ECMAScript : versions clés

Version	Année	Apports majeurs
ES6 / ES2015	2015	`let/const`, arrow fn, classes, modules, Promises
ES2017	2017	`async/await`
ES2018	2018	spread objets, `Promise.all`
ES2019	2019	`Array.flat()`, `Object.fromEntries()`
ES2020	2020	`?.` optional chaining, `??` nullish coalescing
ES2021	2021	`??=` `&&=`, `String.replaceAll()`
ES2022	2022	`Array.at()`, `Object.hasOwn()`
ES2023+	2023+	`Array.toSorted()`, `Array.with()`

2.2 — Features ES Modernes essentielles

```
// Optional chaining (?.)
const user = null
const ville = user?.adresse?.ville      // undefined (pas d'erreur)
const len   = arr?.length ?? 0

// Nullish coalescing (??)
const nom = null ?? "Anonyme"           // "Anonyme"
const age = 0 ?? 18                     // 0 (0 n'est pas nullish)

// Logical assignment
let a = null
a ??= "valeur par défaut"               // a = "valeur par défaut"

// Array.at() — index négatif
const fruits = ["pomme","poire","cerise"]
fruits.at(-1)   // "cerise"
fruits.at(-2)   // "poire"
```

2.3 — Pourquoi TypeScript ?

Sans TypeScript :

```
function calculer(prix, taux) {
  return prix * taux
}
// Appel incorrect — aucune erreur !
calculer("dix", null)
```

Avec TypeScript :

```
function calculer(
  prix: number,
  taux: number
): number {
  return prix * taux
}
// Erreur à la compilation ✅
calculer("dix", null)
// Argument of type 'string' is not
// assignable to parameter 'number'
```

2.4 — TypeScript : types de base

```ts
// Types primitifs
const nom:     string  = "Alice"
const age:     number  = 28
const actif:   boolean = true
const rien:    null    = null
const inconnu: undefined = undefined

// Types composés
const notes:   number[]       = [14, 18, 12]
const tuple:   [string, number] = ["Alice", 28]

// Union de types
let id: string | number = "abc123"
id = 42   // ✅

// Littéraux
type Direction = "nord" | "sud" | "est" | "ouest"
let dir: Direction = "nord"
```

2.5 — TypeScript : interfaces et types

```typescript
// Interface — décrit la forme d'un objet
interface Utilisateur {
  id:       number
  nom:      string
  email:    string
  age?:     number      // propriété optionnelle
  readonly createdAt: Date  // lecture seule
}

// Type alias — plus flexible
type Statut = "actif" | "inactif" | "suspendu"

type Produit = {
  id:    number
  nom:   string
  prix:  number
  stock: boolean
}

// Étendre une interface
interface Admin extends Utilisateur {
  role: "superadmin" | "admin"
  permissions: string[]
}
```

2.6 — TypeScript : fonctions typées

```ts
// Typage complet d'une fonction
function additionner(a: number, b: number): number {
  return a + b
}

// Avec paramètre optionnel
function saluer(nom: string, titre?: string): string {
  return titre ? `${titre} ${nom}` : nom
}

// Avec valeur par défaut
function creerUser(
  nom: string,
  role: string = "visiteur"
): Utilisateur {
  return { id: Date.now(), nom, email: "", role }
}

// Type d'une fonction (callback)
type Callback = (erreur: Error | null, data?: unknown) => void
```

2.7 — TypeScript : generics

```ts
// Generic — type paramétré
function premier<T>(tableau: T[]): T | undefined {
  return tableau[0]
}
premier<number>([1, 2, 3])    // 1
premier<string>(["a", "b"])  // "a"

// Interface générique
interface ApiResponse<T> {
  data:    T
  status:  number
  message: string
}

// Utilisation
const resp: ApiResponse<Utilisateur[]> = await fetchUsers()
resp.data    // Utilisateur[]
resp.status  // number
```

2.8 — TypeScript : utilitaires de types

```ts
interface Utilisateur {
  id: number; nom: string; email: string; age: number
}

// Partial — toutes propriétés optionnelles
type MajUtilisateur = Partial<Utilisateur>

// Required — toutes propriétés requises
type UserComplet = Required<Utilisateur>

// Pick — sélectionner des propriétés
type UserPublic = Pick<Utilisateur, "id" | "nom">

// Omit — exclure des propriétés
type UserSansId = Omit<Utilisateur, "id">

// Readonly — immuable
type UserImmuable = Readonly<Utilisateur>

// Record — dictionnaire typé
type RolesMap = Record<string, string[]>
```

2.9 — Tooling : npm et gestion des paquets

```
# Initialiser un projet
npm init -y

# Installer des dépendances
npm install vue                    # dépendance
npm install -D typescript vitest   # dev seulement

# Commandes utiles
npm run dev        # lancer le serveur
npm run build      # construire pour la prod
npm run test       # lancer les tests
npm run lint       # vérifier le code

# package.json — scripts
{
  "scripts": {
    "dev":   "vite",
    "build": "vue-tsc && vite build",
    "test":  "vitest",
    "lint":  "eslint src --ext .vue,.ts"
  }
}
```

2.10 — ESLint + Prettier

```
# Installation
npm install -D eslint prettier @vue/eslint-config-typescript
```

```
// .eslintrc.json
{
  "extends": [
    "plugin:vue/vue3-recommended",
    "@vue/typescript/recommended"
  ],
  "rules": {
    "vue/multi-word-component-names": "warn",
    "no-console": "warn"
  }
}
```

```
// .prettierrc
```

2.11 — Vue DevTools

Installation :

Extension Chrome / Firefox — "Vue.js devtools"

Onglets principaux :

Onglet	Utilité
Components	Arbre des composants, props, état
Pinia	Stores, état global, actions
Router	Routes actives, historique
Timeline	Événements, mutations dans le temps
Performance	Profiling des rendus

> 💡 Tu peux **modifier l'état** directement depuis DevTools pour tester.

2.12 — Debugging avec les DevTools navigateur

```
// Points d'arrêt dans le code
debugger  // pause l'exécution ici

// Console avancée
console.log("simple")
console.warn("attention")
console.error("erreur")
console.table([{id:1,nom:"Alice"},{id:2,nom:"Bob"}])
console.group("Mon groupe")
  console.log("item 1")
  console.log("item 2")
console.groupEnd()
console.time("ma-mesure")
// ... code à mesurer ...
console.timeEnd("ma-mesure")
```

2.13 — tsconfig.json pour Vue.js

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "jsx": "preserve",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    },
    "types": ["vite/client"]
  },
  "include": ["src/**/*.ts", "src/**/*.vue"],
  "exclude": ["node_modules", "dist"]
}
```

2.14 — Récapitulatif Module 2

ECMAScript

- `?.` `??` `??=`
- `Array.at()`
- Modules ES6

TypeScript

- Types primitifs & composés
- Interfaces & type alias
- Generics & utilitaires
- Fonctions typées

Tooling

- npm / scripts
- ESLint + Prettier

✏️ Exercice — Module 2

Exercice 2.1 — TypeScript : typage de base

Exercice 2.2 — TypeScript : interface et tableau

Exercice 2.3 — DevTools & Debugging

> 🕐 Durée estimée : 40 minutes

Module 3

Utilisation de Vue.js avec Vite

Fondamentaux · Architecture · Scripts

Module 3 — Plan

1. Pourquoi Vite ?
2. Créer un projet Vue.js avec Vite
3. Architecture du projet
4. Fichiers de configuration
5. Scripts et commandes
6. Variables d'environnement
7. Aliasés et imports
8. ✏️ Exercice

3.1 — Vite : pourquoi c'est révolutionnaire ?

Webpack (ancienne génération)

- Bundle tout le code au démarrage
- Démarrage en **30-60 secondes** sur gros projets
- HMR lent

Vite (nouvelle génération)

- Sert les fichiers en **ESModules natifs**
- Démarrage en **< 1 seconde**
- HMR quasi instantané

```
Webpack :  npm run dev → bundle → serve
Vite    :  npm run dev → serve → bundle à la demande
```

3.2 — Créer un projet Vue.js

```
# Méthode recommandée — create-vue (scaffolding officiel)
npm create vue@latest

# Questions du wizard :
✓ Project name:                    mon-projet
✓ Add TypeScript?                  Yes
✓ Add JSX Support?                 No
✓ Add Vue Router?                  Yes
✓ Add Pinia?                       Yes
✓ Add Vitest?                      Yes
✓ Add Cypress for E2E Testing?     No
✓ Add ESLint?                      Yes
✓ Add Prettier?                    Yes

# Lancer le projet
cd mon-projet
npm install
npm run dev
```

3.3 — Structure complète du projet

```
mon-projet/
├── public/                # fichiers statiques (copiés tels quels)
│   └── favicon.ico
├── src/
│   ├── assets/            # images, fonts, styles globaux
│   ├── components/        # composants réutilisables
│   │   └── icons/
│   ├── router/            # configuration Vue Router
│   │   └── index.ts
│   ├── stores/            # stores Pinia
│   │   └── counter.ts
│   ├── views/             # composants-pages (liés aux routes)
│   ├── App.vue            # composant racine
│   └── main.ts            # point d'entrée
├── index.html             # template HTML
├── vite.config.ts         # configuration Vite
├── tsconfig.json          # configuration TypeScript
└── package.json
```

3.4 — Le fichier `main.ts`

```ts
// src/main.ts — point d'entrée de l'application
import { createApp } from 'vue'
import { createPinia } from 'pinia'

import App from './App.vue'
import router from './router'

// Import des styles globaux
import './assets/main.css'

// Créer l'application Vue
const app = createApp(App)

// Installer les plugins
app.use(createPinia())   // gestionnaire d'état
app.use(router)          // navigation

// Monter sur l'élément #app dans index.html
app.mount('#app')
```

3.5 — `App.vue` — Le composant racine

```
<!-- src/App.vue -->
<template>
  <header>
    <nav>
      <RouterLink to="/">Accueil</RouterLink>
      <RouterLink to="/about">À propos</RouterLink>
    </nav>
  </header>

  <main>
    <!-- Les vues sont injectées ici par Vue Router -->
    <RouterView />
  </main>
</template>

<script setup lang="ts">
// Pas besoin d'importer RouterLink et RouterView,
// ils sont auto-importés par Vue Router
</script>

<style scoped>
header { padding: 1rem; background: #f0f0f0; }
</style>
```

3.6 — `vite.config.ts`

```ts
import { fileURLToPath, URL } from 'node:url'
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],

  resolve: {
    alias: {
      // '@' pointe vers 'src/' — import propres
      '@': fileURLToPath(new URL('./src', import.meta.url))
    }
  },

  server: {
    port: 3000,
    proxy: {
      // Proxy l'API pour éviter les problèmes CORS en dev
      '/api': 'http://localhost:8080'
    }
  }
})
```

3.7 — Scripts NPM et leur rôle

```
{
  "scripts": {
    "dev":       "vite",
    "build":     "vue-tsc && vite build",
    "preview":   "vite preview",
    "test:unit": "vitest",
    "lint":      "eslint . --ext .vue,.js,.jsx,.cjs,.mjs,.ts --fix"
  }
}
```

Commande	Rôle
`npm run dev`	Serveur de développement + HMR
`npm run build`	Compile TS + bundle pour la prod
`npm run preview`	Prévisualise le build de prod

3.8 — Variables d'environnement

```
# .env                  — toujours chargé
# .env.local            — chargé, ignoré par git
# .env.development      — mode dev uniquement
# .env.production       — mode prod uniquement

# Préfixe VITE_ obligatoire pour être exposé au client
VITE_API_URL=https://api.example.com
VITE_APP_TITLE=Mon Application
```

```
// Utilisation dans le code
const apiUrl   = import.meta.env.VITE_API_URL
const appTitle = import.meta.env.VITE_APP_TITLE
const isProd   = import.meta.env.PROD      // boolean
const isDev    = import.meta.env.DEV       // boolean
const mode     = import.meta.env.MODE      // "development" | "production"
```

3.9 — Imports avec l'alias `@`

```
// ❌ Sans alias — chemin relatif fragile
import MonComposant from '../../components/MonComposant.vue'
import { useAuth }  from '../../../stores/auth'

// ✅ Avec alias @ — toujours depuis src/
import MonComposant from '@/components/MonComposant.vue'
import { useAuth }  from '@/stores/auth'
import router       from '@/router'
import '@/assets/styles/global.css'
```

> L'alias `@` est configuré dans `vite.config.ts` et `tsconfig.json`.

3.10 — Hot Module Replacement (HMR)

```
Modification d'un fichier
        ↓
Vite détecte le changement
        ↓
Seul le module modifié est recompilé
        ↓
Le navigateur reçoit la mise à jour
        ↓
L'état de l'application est conservé ✅
```

Avantages du HMR Vite :

- Mise à jour en < **50ms** même sur gros projets
- L'état des composants est **préservé** entre les sauvegardes
- Pas de rechargement complet de la page

3.11 — Récapitulatif Module 3

Concept	Commande / Fichier
Créer un projet	`npm create vue@latest`
Démarrer	`npm run dev`
Builder	`npm run build`
Config Vite	`vite.config.ts`
Config TS	`tsconfig.json`
Alias	`@` → `src/`
Env variables	`VITE_*` dans `.env`
Point d'entrée	`src/main.ts`

✏️ Exercice — Module 3

Exercice 3.1 — Scaffold du projet

Exercice 3.2 — Exploration de la structure

Exercice 3.3 — Scripts NPM

> 🕐 Durée estimée : 30 minutes

Module 4

Les Composants Vue.js

SFC · Templating · Props · Emit · Lifecycle

Module 4 — Plan

1. Single File Components (SFC)
2. Syntaxe de template
3. Directives fondamentales
4. Props et validation
5. Émission d'événements
6. Slots
7. Hooks de cycle de vie
8. Composition avancée
9. ✏️ Exercice

4.1 — Single File Component (SFC)

```
<!-- MonComposant.vue — les 3 blocs -->

<template>
  <!-- HTML du composant -->
  <div class="carte">
    <h2>{{ titre }}</h2>
    <p>{{ description }}</p>
  </div>
</template>

<script setup lang="ts">
// Logique JavaScript / TypeScript
defineProps<{ titre: string; description: string }>()
</script>

<style scoped>
/* CSS scopé à ce composant uniquement */
.carte { border: 1px solid #ddd; padding: 1rem; }
</style>
```

4.2 — Interpolation et expressions

```
<template>
  <!-- Interpolation de texte -->
  <p>{{ message }}</p>
  <p>{{ message.toUpperCase() }}</p>
  <p>{{ nombre * 2 }}</p>
  <p>{{ condition ? "Oui" : "Non" }}</p>

  <!-- Liaison d'attributs (v-bind) -->
  <img :src="imageUrl" :alt="imageAlt" />
  <button :disabled="estDesactive">Clic</button>
  <div :class="{ actif: estActif, erreur: aErreur }"></div>
  <div :style="{ color: couleur, fontSize: taille + 'px' }"></div>

  <!-- HTML brut (attention XSS !) -->
  <div v-html="htmlBrut"></div>
</template>
```

4.3 — Directives essentielles

```
<template>
  <!-- v-if / v-else-if / v-else — rendu conditionnel -->
  <div v-if="utilisateur.role === 'admin'">Panneau admin</div>
  <div v-else-if="utilisateur.role === 'modo'">Panneau modo</div>
  <div v-else>Accès limité</div>

  <!-- v-show — affichage CSS (pas de suppression du DOM) -->
  <div v-show="panneauVisible">Contenu</div>

  <!-- v-for — rendu de liste -->
  <ul>
    <li v-for="item in items" :key="item.id">
      {{ item.nom }}
    </li>
  </ul>

  <!-- v-for avec index -->
  <li v-for="(item, index) in items" :key="item.id">
    {{ index + 1 }}. {{ item.nom }}
  </li>
</template>
```

4.4 — v-model : liaison bidirectionnelle

```
<template>
  <!-- v-model sur un input -->
  <input v-model="prenom" placeholder="Votre prénom" />
  <p>Bonjour {{ prenom }}</p>

  <!-- v-model sur différents types -->
  <input type="checkbox" v-model="accepteCgu" />
  <input type="radio" v-model="genre" value="M" />
  <select v-model="pays">
    <option value="fr">France</option>
    <option value="be">Belgique</option>
  </select>

  <!-- Modifiers -->
  <input v-model.trim="nom" />      <!-- trim les espaces -->
  <input v-model.number="age" />    <!-- convertit en number -->
  <input v-model.lazy="texte" />    <!-- update au blur -->
</template>
```

4.5 — Props : passage de données parent → enfant

```
<!-- Enfant : CarteUtilisateur.vue -->
<script setup lang="ts">
// Déclaration des props avec types TypeScript
const props = defineProps<{
  nom:     string
  age:     number
  role?:   string       // optionnel
  actif?:  boolean
}>()

// Avec valeurs par défaut
const props = withDefaults(
  defineProps<{ nom: string; age?: number }>(),
  { age: 18 }
)
</script>

<template>
  <div>
    <h3>{{ props.nom }}</h3>
    <span>{{ props.age }} ans</span>
    <span v-if="props.role">{{ props.role }}</span>
  </div>
</template>
```

4.6 — Props : utilisation dans le parent

```
<!-- Parent : App.vue -->
<template>
  <!-- Valeur statique -->
  <CarteUtilisateur nom="Alice" :age="28" />

  <!-- Valeur dynamique -->
  <CarteUtilisateur
    :nom="utilisateur.nom"
    :age="utilisateur.age"
    :role="utilisateur.role"
    :actif="true"
  />

  <!-- Spread d'un objet -->
  <CarteUtilisateur v-bind="utilisateur" />
</template>

<script setup lang="ts">
import CarteUtilisateur from '@/components/CarteUtilisateur.vue'
const utilisateur = { nom: "Bob", age: 32, role: "admin" }
</script>
```

4.7 — Emit : communication enfant → parent

```
<!-- Enfant : Formulaire.vue -->
<script setup lang="ts">
// Déclarer les événements émis
const emit = defineEmits<{
  submit: [data: { nom: string; email: string }]
  annuler: []
}>()

function soumettre() {
  emit('submit', { nom: formNom.value, email: formEmail.value })
}
</script>

<!-- Parent : App.vue -->
<template>
  <Formulaire
    @submit="handleSubmit"
    @annuler="panneauVisible = false"
  />
</template>

<script setup lang="ts">
function handleSubmit(data: { nom: string; email: string }) {
  console.log("Données reçues :", data)
}
</script>
```

4.8 — Slots : contenu injecté par le parent

```
<!-- Enfant : Carte.vue -->
<template>
  <div class="carte">
    <header class="carte-header">
      <slot name="header">Titre par défaut</slot>
    </header>
    <main class="carte-body">
      <slot></slot>  <!-- slot par défaut -->
    </main>
    <footer class="carte-footer">
      <slot name="footer"></slot>
    </footer>
  </div>
</template>

<!-- Parent -->
<Carte>
  <template #header>🎉 Mon titre personnalisé</template>
  <p>Contenu principal de la carte</p>
  <template #footer>
    <button>Action</button>
  </template>
</Carte>
```

4.9 — Hooks de cycle de vie

```
import { onMounted, onUpdated, onUnmounted, onBeforeMount } from 'vue'

// Ordre d'exécution :
// setup() → onBeforeMount → [DOM créé] → onMounted
// [données changent] → onBeforeUpdate → [DOM mis à jour] → onUpdated
// [composant détruit] → onBeforeUnmount → onUnmounted

onMounted(() => {
  // ✅ DOM disponible — idéal pour :
  // - fetch de données initiales
  // - initialiser des libs tierces
  // - accéder aux refs du DOM
  console.log("Composant monté")
})

onUnmounted(() => {
  // Nettoyage — important pour éviter les fuites mémoire
  clearInterval(timer)
  eventBus.off("mon-event")
})
```

4.10 — Template refs — accès direct au DOM

```
<template>
  <input ref="champNom" type="text" />
  <canvas ref="monCanvas"></canvas>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue'

// La ref doit avoir le même nom que l'attribut ref=""
const champNom  = ref<HTMLInputElement | null>(null)
const monCanvas = ref<HTMLCanvasElement | null>(null)

onMounted(() => {
  // DOM disponible après onMounted
  champNom.value?.focus()

  const ctx = monCanvas.value?.getContext('2d')
  ctx?.fillRect(0, 0, 100, 100)
})
</script>
```

4.11 — `defineExpose` — exposer des méthodes

```
<!-- Enfant : ComposantModal.vue -->
<script setup lang="ts">
import { ref } from 'vue'

const visible = ref(false)

function ouvrir()  { visible.value = true }
function fermer()  { visible.value = false }

// Exposer des méthodes au parent via template ref
defineExpose({ ouvrir, fermer })
</script>

<!-- Parent -->
<script setup lang="ts">
import { ref } from 'vue'
const modal = ref()

function afficherModal() {
  modal.value.ouvrir()  // appel direct de la méthode
}
</script>
<template>
  <ComposantModal ref="modal" />
  <button @click="afficherModal">Ouvrir</button>
</template>
```

4.12 — Gestion des événements DOM

```
<template>
  <!-- Événements natifs -->
  <button @click="handleClick">Clic</button>
  <input  @input="handleInput" @keyup.enter="valider" />
  <form   @submit.prevent="soumettre">...</form>

  <!-- Modifiers d'événements -->
  <button @click.stop="fn">Stop propagation</button>
  <div    @click.self="fn">Seulement sur ce div</div>
  <input  @keyup.enter="fn">Enter seulement</input>
  <a      @click.prevent="fn">Empêche la navigation</a>

  <!-- Modifiers de touches -->
  <input @keyup.ctrl.enter="fn" />  <!-- Ctrl + Entrée -->
  <input @keyup.esc="fn" />         <!-- Échap -->
</template>
```

4.13 — Composants asynchrones

```
import { defineAsyncComponent } from 'vue'

// Chargement lazy d'un composant lourd
const GrosComposant = defineAsyncComponent(
  () => import('./composants/GrosComposant.vue')
)

// Avec options avancées
const ComposantAvecFallback = defineAsyncComponent({
  loader:         () => import('./Lourd.vue'),
  loadingComponent: Spinner,    // affiché pendant le chargement
  errorComponent:   Erreur,     // affiché en cas d'échec
  delay:          200,          // délai avant d'afficher le spinner
  timeout:        3000          // timeout après 3s
})
```

💡 Utile pour les modales, onglets, sections rarement visitées.

4.14 — Bonnes pratiques des composants

Nommage :

```
✅ MonComposant.vue        (PascalCase)
✅ carte-utilisateur.vue   (kebab-case)
❌ moncomposant.vue
❌ Mon_Composant.vue
```

Découpage :

- 1 composant = 1 responsabilité
- < 200 lignes dans le `<script>`
- < 100 lignes dans le `<template>`

Clés sur v-for :

4.15 — Récapitulatif Module 4

SFC

- `<template>` `<script setup>` `<style scoped>`
- Interpolation `{{ }}`
- `:attr` `@event`

Directives

- `v-if` / `v-show` / `v-for`
- `v-model` + modifiers
- `v-bind` / `v-on`

Communication

- `defineProps<{}>()` parent→enfant
- `defineEmits<{}>()` enfant→parent

✏️ Exercice — Module 4

Exercice 4.1 — Premier composant

Exercice 4.2 — Communication parent / enfant

Exercice 4.3 — Décomposition d'une interface

Exercice 4.4 — Cycle de vie

> 🕐 Durée estimée : 1h00

🎯 Bilan Jour 1

Ce que vous savez faire

- ✅ JavaScript moderne (ES6+, async/await)
- ✅ TypeScript — types, interfaces, generics
- ✅ Créer un projet Vue.js avec Vite
- ✅ Créer des composants SFC complets
- ✅ Gérer la communication parent ↔️ enfant
- ✅ Utiliser les directives Vue.js
- ✅ Comprendre le cycle de vie

Demain — Jour 2

📋 Index des slides — Jour 1

Slide	Contenu
1	Couverture
2	Sommaire
3	Module 1 — chapitre
4–19	JavaScript bases
20	Récap Module 1
21	Exercice Module 1
22	Module 2 — chapitre
23–37	ECMAScript, TypeScript, Tooling
38	Récap Module 2

1.16 — Promesses en chaîne

```
// Chaîner les promesses
fetch('/api/token')
  .then(r => r.json())
  .then(({ token }) => fetch('/api/users', {
    headers: { Authorization: `Bearer ${token}` }
  }))
  .then(r => r.json())
  .then(users => console.log(users))
  .catch(err => console.error('Erreur :', err))
  .finally(() => console.log('Terminé'))

// Promise.all — attendre plusieurs promesses
const [users, posts, comments] = await Promise.all([
  fetch('/api/users').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
  fetch('/api/comments').then(r => r.json()),
])

// Promise.allSettled — même si certaines échouent
const results = await Promise.allSettled([p1, p2, p3])
results.forEach(r => r.status === 'fulfilled' && console.log(r.value))
```

1.17 — Erreurs et try/catch

```
// Try / catch / finally
async function chargerUtilisateur(id) {
  try {
    const res = await fetch(`/api/users/${id}`)
    if (!res.ok) {
      throw new Error(`HTTP ${res.status} : ${res.statusText}`)
    }
    return await res.json()
  } catch (error) {
    if (error instanceof TypeError) {
      console.error('Erreur réseau :', error.message)
    } else {
      console.error('Erreur API :', error.message)
    }
    return null
  } finally {
    // S'exécute TOUJOURS — succès ou erreur
    console.log('Requête terminée')
  }
}
```

1.18 — Classes JavaScript

```
class Animal {
  #nom  // propriété privée
  constructor(nom, espece) {
    this.#nom   = nom
    this.espece = espece
  }
  get nom() { return this.#nom }  // getter
  parler() { return `${ this.#nom} fait un son` }
}

class Chien extends Animal {
  constructor(nom) {
    super(nom, 'Canis lupus')
  }
  parler() {
    return `${ super.parler()} : Wouf !`
  }
  static creer(nom) { return new Chien(nom) }  // méthode statique
}

const rex = Chien.creer('Rex')
rex.parler()  // "Rex fait un son : Wouf !"
```

1.19 — Iterateurs et Generators

```
// Symbol.iterator — rendre un objet itérable
class Plage {
  constructor(debut, fin) {
    this.debut = debut; this.fin = fin
  }
  [Symbol.iterator]() {
    let current = this.debut
    const fin   = this.fin
    return {
      next() {
        return current <= fin
          ? { value: current++, done: false }
          : { done: true }
      }
    }
  }
}

for (const n of new Plage(1, 5)) console.log(n)  // 1,2,3,4,5

// Generator
function* compter(debut = 0) {
  while (true) yield debut++
}
const gen = compter(10)
gen.next().value  // 10
gen.next().value  // 11
```

1.20 — Map, Set et WeakMap

```
// Map — clés de n'importe quel type
const scores = new Map()
scores.set('Alice', 95)
scores.set('Bob',   87)
scores.get('Alice')        // 95
scores.has('Carol')        // false
scores.size                // 2

for (const [nom, score] of scores) {
  console.log(`${nom}: ${score}`)
}

// Set — valeurs uniques
const tags = new Set(['vue', 'js', 'vue', 'ts'])
tags.size  // 3 (doublon supprimé)
tags.add('react')

// Supprimer les doublons d'un tableau
const unique = [...new Set([1,1,2,3,3,4])]  // [1,2,3,4]
```

1.21 — WeakMap et gestion mémoire

```
// WeakMap — clés doivent être des objets, garbage-collectable
const cache = new WeakMap()

function calculerEnCache(obj) {
  if (cache.has(obj)) return cache.get(obj)
  const result = computeExpensive(obj)
  cache.set(obj, result)
  return result
}
// Quand 'obj' est garbage-collecté, l'entrée du WeakMap l'est aussi
// → Pas de fuite mémoire ✅

// WeakSet — similaire pour les ensembles
const vus = new WeakSet()
function processer(obj) {
  if (vus.has(obj)) return
  vus.add(obj)
  // ... traitement
}
```

1.22 — Récapitulatif Module 1 — Avancé

Concept	Méthode / Syntaxe
Promesses	`.then()` `.catch()`, `Promise.all()`
Async	`async function`, `await`, `try/catch`
Classes	`class`, `extends`, `super`, `#private`
Itération	`for...of`, `Symbol.iterator`, `function*`
Collections	`Map`, `Set`, `WeakMap`, `WeakSet`

💡 Ces concepts reviennent constamment dans Vue.js :

- `async/await` dans les actions Pinia
- `Map/Set` dans les computed properties
- Classes dans les services et utils

3.12 — Plugins Vite utiles

```
// vite.config.ts — plugins courants
import { defineConfig }         from 'vite'
import vue                      from '@vitejs/plugin-vue'
import vueDevTools              from 'vite-plugin-vue-devtools'
import AutoImport               from 'unplugin-auto-import/vite'
import Components               from 'unplugin-vue-components/vite'

export default defineConfig({
  plugins: [
    vue(),
    vueDevTools(),  // DevTools améliorées

    // Auto-import de ref, computed, etc.
    AutoImport({ imports: ['vue', 'vue-router', 'pinia'] }),

    // Import automatique des composants du dossier components/
    Components({ dirs: ['src/components'] }),
  ]
})
```

3.13 — Path aliases multiples

```
// vite.config.ts — plusieurs alias utiles
import { fileURLToPath, URL } from 'node:url'

export default defineConfig({
  resolve: {
    alias: {
      '@':           fileURLToPath(new URL('./src', import.meta.url)),
      '@components': fileURLToPath(new URL('./src/components', import.meta.url)),
      '@stores':     fileURLToPath(new URL('./src/stores', import.meta.url)),
      '@views':      fileURLToPath(new URL('./src/views', import.meta.url)),
      '@utils':      fileURLToPath(new URL('./src/utils', import.meta.url)),
      '@types':      fileURLToPath(new URL('./src/types', import.meta.url)),
    }
  }
})

// Utilisation :
import { useAuth } from '@stores/auth'
import UserCard    from '@components/UserCard.vue'
import type { User } from '@types/user'
```

4.16 — Composants fonctionnels

```
// Composant fonctionnel — sans état, ultra-léger
// Idéal pour des éléments purement d'affichage

import type { FunctionalComponent } from 'vue'

interface Props { label: string; type?: 'primary' | 'secondary' }

const Badge: FunctionalComponent<Props> = (props, { slots }) => {
  return h('span', {
    class: ['badge', `badge--${props.type ?? 'primary'}`]
  }, props.label)
}

Badge.props = {
  label: { type: String, required: true },
  type:  { type: String, default: 'primary' }
}

// Avantages des composants fonctionnels :
// - Pas d'instance Vue créée
// - Rendu plus rapide
// - Idéal pour des milliers d'éléments dans une liste
```

4.17 — Composants d'ordre supérieur (HOC)

```
// HOC — wrapper qui ajoute des fonctionnalités
function withLoading<T>(WrappedComponent: Component) {
  return defineComponent({
    props: { loading: Boolean, ...WrappedComponent.props },
    setup(props, { attrs, slots }) {
      return () => props.loading
        ? h('div', { class: 'skeleton' }, 'Chargement...')
        : h(WrappedComponent, { ...props, ...attrs }, slots)
    }
  })
}

// Utilisation
const UserCardWithLoading = withLoading(UserCard)

// Dans le template
<UserCardWithLoading
  :loading="isLoading"
  :nom="user.nom"
  :age="user.age"
/>
```

4.18 — Validation des props avec runtime checks

```
defineProps({
  age: {
    type:     Number,
    required: true,
    validator: (value: number) => {
      if (value < 0 || value > 150) {
        console.warn(`Age invalide : ${value}`)
        return false
      }
      return true
    }
  },
  status: {
    type:    String,
    default: 'actif',
    validator: (val: string) =>
      ['actif', 'inactif', 'suspendu'].includes(val)
  },
  config: {
    type:    Object as PropType<AppConfig>,
    default: () => ({})   // ⚠️ Toujours une factory pour objets/tableaux
  }
})
```

4.19 — Directives personnalisées

```
// Directive v-focus — met le focus automatiquement
const vFocus = {
  mounted(el: HTMLElement) { el.focus() }
}

// Directive v-tooltip
const vTooltip = {
  mounted(el: HTMLElement, binding: DirectiveBinding) {
    el.title = binding.value
    el.style.cursor = 'help'
  },
  updated(el: HTMLElement, binding: DirectiveBinding) {
    el.title = binding.value
  }
}

// Enregistrement global
app.directive('focus',   vFocus)
app.directive('tooltip', vTooltip)

// Utilisation dans le template
<input v-focus />
<span v-tooltip="'Cliquer pour plus d\'infos'">?</span>
```

4.20 — Plugin Vue personnalisé

```
// plugins/toast.ts — plugin de notifications
import type { App } from 'vue'

const toastPlugin = {
  install(app: App, options = {}) {
    // Ajouter une propriété globale
    app.config.globalProperties.$toast = {
      success: (msg: string) => showToast(msg, 'success'),
      error:   (msg: string) => showToast(msg, 'error'),
      info:    (msg: string) => showToast(msg, 'info'),
    }

    // Ajouter un composant global
    app.component('AppToast', ToastComponent)

    // Provide pour inject
    app.provide('toast', app.config.globalProperties.$toast)
  }
}

// main.ts
app.use(toastPlugin, { duration: 3000 })

// Dans un composant
const toast = inject('toast')
toast.success('Sauvegardé !')
```

2.15 — Écrire du TypeScript idiomatique

```
// Éviter any — utiliser unknown pour les types inconnus
function traiter(valeur: unknown) {
  if (typeof valeur === 'string') {
    console.log(valeur.toUpperCase())  // type narrowing ✅
  }
}

// Assertion de type — seulement si on est certain
const input = document.getElementById('email') as HTMLInputElement
input.value  // accessible ✅

// Type guards personnalisés
function estUtilisateur(obj: unknown): obj is Utilisateur {
  return typeof obj === 'object' && obj !== null && 'nom' in obj
}

if (estUtilisateur(data)) {
  console.log(data.nom)  // TypeScript sait que c'est un Utilisateur
}
```

2.16 — Mapped Types et Template Literal Types

```ts
// Mapped types — transformer les propriétés d'un type
type Optional<T> = { [K in keyof T]?: T[K] }
type Nullable<T> = { [K in keyof T]: T[K] | null }
type ReadOnly<T> = { readonly [K in keyof T]: T[K] }

// Template literal types
type EventName = 'click' | 'focus' | 'blur'
type Handler = `on${Capitalize<EventName>}`
// → 'onClick' | 'onFocus' | 'onBlur'

// Pratique pour des APIs fortement typées
type CssProperty = `--${ string }`
const custom: CssProperty = '--ma-couleur'  // ✅

// Conditional types
type IsArray<T> = T extends any[] ? 'tableau' : 'pas un tableau'
type R1 = IsArray<string[]>  // 'tableau'
type R2 = IsArray<number>    // 'pas un tableau'
```

3.14 — Gestion des assets statiques

```
// Import d'images en TypeScript
import logoUrl from '@/assets/logo.png'
// → URL résolue avec hash en prod

// Import dynamique d'images
const modules = import.meta.glob('@/assets/icons/*.svg', {
  eager: true,
  import: 'default'
})
// { '@/assets/icons/home.svg': 'data:...', ... }

// Référence dans le template
<img :src="logoUrl" alt="Logo" />

// URL publique (sans import)
// Fichiers dans /public/ accessibles par chemin direct
<img src="/images/hero.jpg" />

// import.meta.env.BASE_URL — utile pour les sous-chemins
const url = `${import.meta.env.BASE_URL}images/hero.jpg`
```

4.21 — `v-bind` dynamique et multi-attributs

```
<template>
  <!-- Lier un objet entier d'attributs -->
  <input v-bind="inputAttrs" />

  <!-- Classes dynamiques avancées -->
  <div :class="[
    'base-class',
    { 'is-active': isActive },
    { 'has-error': hasError },
    conditionClass
  ]"></div>

  <!-- Style dynamique avancé -->
  <div :style="[baseStyle, conditionalStyle]"></div>
</template>

<script setup>
const inputAttrs = reactive({
  type:        'email',
  placeholder: 'votre@email.com',
  autocomplete:'email',
  required:    true
})

const baseStyle       = { fontSize: '14px', color: '#333' }
const conditionalStyle = computed(() =>
  isError.value ? { borderColor: 'red' } : {}
)
</script>
```

4.22 — Gestion des formulaires complexes

```
<script setup lang="ts">
import { reactive } from 'vue'

// État centralisé du formulaire
const form = reactive({
  prenom:  '',
  nom:     '',
  email:   '',
  age:     null as number | null,
  role:    'user',
  tags:    [] as string[],
  adresse: { rue: '', ville: '', cp: '' }
})

// Erreurs de validation
const errors = reactive<Record<string, string>>({})

function valider() {
  errors.email = ''
  if (!form.email.includes('@')) {
    errors.email = 'Email invalide'
  }
  return Object.keys(errors).every(k => !errors[k])
}

async function soumettre() {
  if (!valider()) return
  await api.post('/users', form)
}
</script>
```